



第七章 概率算法

Chapter 7 Probabilistic Algorithm

计算机学院 潘永生

- ***学习要点:***
  - 理解产生伪随机数的算法
  - 掌握舍伍德算法的设计思想
  - 掌握拉斯维加斯算法的设计思想
  - 掌握蒙特卡罗算法的设计思想

概率算法设计思想

- 在多数情况下，当算法在执行过程中面临一个选择时，随机性选择常比最优选择省时。因此，概率算法可在很大程度上降低算法复杂性。
- 概率算法放宽了“对于所有合理的输入都必须给出正确的输出”这一求解问题的条件，把随机性的选择注入到算法中。
  - 在算法执行某些步骤时，可以随机地选择下一步该如何进行，同时允许结果以较小的概率出现错误，并以此为代价，获得算法运行时间的大幅度减少。

- 概率算法基本特征：
  - 概率算法的*输入*：
    - 原问题的输入
    - 供算法进行随机选择的随机数序列
  - *运行过程*中：
    - 一处或若干处随机选择，根据随机值来决定算法的运行
  - *结果*：
    - 不能保证一定是正确的
    - 可以限定其出错概率
  - *不同的运行*中，对于相同的输入实例可以有不同的结果。
    - 因此，对于相同的输入实例，概率算法的执行时间可能不同。
- 概率算法的时间复杂性：

  期望时间复杂性——由概率算法反复运行同一输入实例所得的平均运行时间。

- 随机数在概率算法设计中扮演着十分重要的角色。在现实计算机上无法产生真正的随机数，因此在概率算法中使用的随机数都是一定程度上随机的，即伪随机数。
- 线性同余法是产生伪随机数的最常用的方法。由线性同余法产生的随机序列$a_0, a_1, \ldots, a_n$满足：

$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \mod m \qquad n = 1,2,\cdots \end{cases}$$

- 其中，$b \geq 0$，$c \geq 0$，$m > 0$，$d \leq m$。d称为随机数发生器的随机种子(Random Seed)，当b、c和m的值确定后，给定一个随机种子，由上式产生的随机数序列也就确定了。从直观上看，m应取的充分大，因此可取m为机器大数；另外应取gcd(m,b)=1，因此可取b为一素数。

- 伪随机数产生：
  - Random(n)---产生0 ~ n-1范围内的随机整数
  - LRandom(L, r)---产生L~r范围内的随机整数
- 概率算法是一般算法随机化后的结果，它没有扩大原有的可计算函数的范围，所以不会影响已有的可计算性概念和结果。
- 设计概率算法的步骤如下：
  - 对给定问题设计一个确定算法A；
  - 分析算法A的各个步骤及每一步骤的有关操作，以选定随机化的入口和随机化方式，从而将算法A随机化，得到概率算法A’；
  - 论证A’是一个好的算法，即证明：A’的正确性和A’比A更好。

利用线性同余式计算新的种子，将randSeed右移16位得到一个0~65535间的随机数，然后再将此随机整数映射到0~(n-1)范围内。

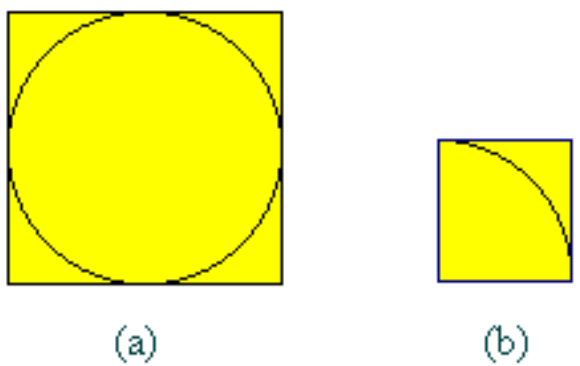
```
Random（n, s)
{ //产生 0 : n-1之间的随机整数，s为随机数发生器种子
if( s==0)
randSeed= time(0); //用系统时间产生种子
else
randSeed = s; //由用户提供种子
randSeed = multiplier * randSeed + adder;
return (randSeed >>16) % n ;
}
```

其中： multiplier = 1194211693L, adder = 12345L

用随机投点法计算π值

设有一半径为r的圆及其外切四边形。向该正方形随机地投掷n个点。设落入圆内的点数为k。由于所投入的点在正方形上均匀分布，因而所投入的点落入圆内的概率为$\frac{\pi r^2}{4r^2}=\frac{\pi}{4}$。所以当n足够大时，k与n之比就逼近这一概率。从而 $\pi \approx \frac{4k}{n}$。

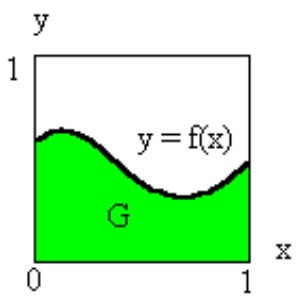
```
public static double darts(int n)
  { // 用随机投点法计算π值
    int k=0;
    for (int i=1;i <=n;i++) {
      double x=dart.fRandom();
      double y=dart.fRandom();
      if ((x*x+y*y)<=1) k++;
      }
    return 4*k/(double)n;
  }
```



计算定积分

设f(x)是[0，1]上的连续函数，且0≤f(x)≤1。

需要计算的积分为 $I=\int_0^1 f(x)\,dx$,积分I等于图中的面积G。



在图所示单位正方形内均匀地作投点试验，则随机点落在曲线下面的概率为

$$P_r\{y \le f(x)\} = \int_0^1 \int_0^{f(x)} dydx = \int_0^1 f(x)\,dx$$

假设向单位正方形内随机地投入n个点(xi,yi)。如果有m个点落入G内，则随机点落入G内的概率 $I \approx \frac{m}{n}$

```
public static double darts(int n)
    { // 用随机投点法计算定积分
      int k=0;
      for (int i=1;i <=n;i++) {
        double x=dart.fRandom();
        double y=dart.fRandom();
        if (y<=f(x)) k++;
        }
      return k/(double)n;
    }
```

特点

- Sherwood算法总能求得问题的一个正确解；
- Sherwood算法能消除算法最坏特性与平均特性之间的差别。

  设：一个确定型算法A，它求解问题规模为n的所有输入实例的平均时间是$T_A(n)$。

  而一个Sherwood型随机算法B，它求解问题规模为n的输入实例的时间是$T_B(n)$。

  一般地，有：$T_B(n) = T_A(n) + s(n)$

∵ Sherwood型随机算法B本身的随机特性

∴ 对于问题的某一输入实例，有：$T_B(n) > T_A(n)$

但，其原因是随机算法本身的随机性，而不在于问题的实例！

设：Sherwood型随机算法B的平均时间是$T_B(n)$。

则有：$T_B(n) = T_A(n) + s(n)$ 其中，$s(n) << T_A(n)$

- 舍伍德型概率算法不是避免算法的最坏情况行为，而是设法消除算法的不同输入实例对算法时间性能的影响。
- 对所有输入实例而言，舍伍德型概率算法的运行时间相对比较均匀。

- 快速排序的概率算法

例7-1:快速排序。

- 关键在于一次划分中选择合适的轴值作为划分的基准。
  - 当输入数据是均匀分布时，快速排序算法所需的平均时间是$O(n\log_2 n)$;
  - 输入已经基本排好序时，所用时间就大大增加了，最坏是$O(n^2)$ 。
- 采用舍伍德算法消除算法所需计算时间与输入实例间的这种联系。

□ 快速排序的舍伍德型算法思想：

初始键值序列　6 13 19 23 31 35 58

i ↑　　↑ j

一次划分结果　6 [13 19 23 31 35 58]

(调用Partition)

(a) 以最小值6为轴值，划分不均衡

初始键值序列　6 13 19 23 31 35 58

随机选择一个记录与6交换　23 13 19 6 31 35 58

i ↑　　↑ j

一次划分结果　[6 13 19] 23 [31 35 58]

(调用Partition)

(b) 随机选择轴值，划分均衡

□ 算法描述：

```
void  R_QuickSort (int r[ ], int low, int high)
{
    if (low<high) {
        i=LRandom(low, high); //在区间[low, high]中随机选取一个元素
        r[low]←→r[i]; //交换元素
        k=Partition(r, low, high); //进行一次划分
        R_QuickSort(r, low, k-1);  //对左边子序列进行快排
        R_QuickSort(r, k+1, high); //对右边子序列进行快排
    }
}
```

复杂度分析：

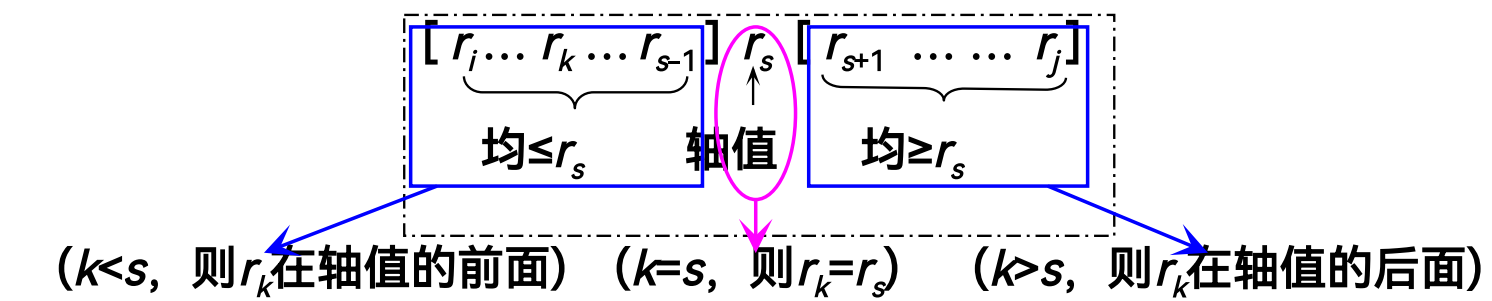
✔最坏情况下，$T(n)=O(n^2)$ —— 小概率

✔平均复杂度，$T(n)=O(n\log_2 n)$

K-选择的概率算法

例7-2:设无序序列$T=(r_1, r_2, \ldots, r_n)$，$T$的第$k(1\le k\le n)$小元素定义为$T$按升序排列后在第$k$个位置上的元素。给定一个序列$T$和一个整数$k$，寻找$T$的第$k$小元素的问题称为K-选择问题。

- 选定一个轴值将序列$r_i \sim r_j$进行划分，使得比轴值小的元素都位于轴值的左侧，比轴值大的元素都位于轴值的右侧，假定轴值的最终位置是s，则：



- K-选择的舍伍德型算法思想：

  在一次划分之前，根据随机数在待划分序列中随机确定一个记录作为轴值，并把它与第一个记录交换，则一次划分后得到期望均衡的两个子序列，使得选择问题在最坏情况下的时间性能趋近于平均情况的时间性能。

```
int R_K-Select(int r[ ], int low, int high, int k)
{
   if (high-low<=k) return r[high];  //数组长度小于k
   else {
    i=LRandom(low, high);    //在区间[low, high]中随机选取一个元素
    r[low]←→r[i];          //交换元素
    s=Partition(r, low, high);  //进行一次划分，得到轴值的位置s
    if (k= =s) return r[s];     //元素r[s]就是第k小元素
    else if (k>s) return R_K-Select(r, low, s-1, k);  //在前半部分继续查找
         else return R_K-Select(r, s+1, high, k);   //在后半部分继续查找
   }
```

- 随机预处理技术
  - 不改变原有的确定性算法，仅对其输入进行随机洗牌，同样可以得到舍伍德算法的效果。
  - 假设输入实例为整型，下面的随机洗牌算法可在线性时间实现对输入实例的随机排列。

```
void RandomShuffle(int n, int r[ ])
{
    for (i=0; i<n; i++)
   {
       j=Random(n-i);
       r[i]←→r[j];      //交换r[i]和r[j]
   }
}
```

特点

- Las Vegas算法有时找不到问题实例的解，即，Las Vegas算法可能运行失败!
- Las Vegas算法所能找到的解是正确的;
- 拉斯维加斯型概率算法的返回类型为bool;
- 有两个参数:
  - 算法的输入
  - 当算法运行成功时保存问题的解

拉斯维加斯算法不会得到不正确的解。一旦用拉斯维加斯算法找到一个解，这个解就一定是正确解。但有时用拉斯维加斯算法会找不到解。拉斯维加斯算法找到正确解的概率随着它所用的计算时间的增加而提高。对于所求解的任一实例，用同一拉斯维加斯算法反复对该例求解足够多次，可使求解失效的概率任意小。　通常用一个bool类型的函数表示拉斯维加斯算法，拉斯维加斯算法的典型调用形式为bool success=LV(x,y)；当success值为false时，算法未能找到问题的解。此时可对同一实例再次独立地调用相同的算法。

- 设：bool LV(x, y)是求解问题实例x的Las Vegas算法的程序实现。则，Las Vegas算法在程序实现时，应包含代码：

  while (! LV(x, y));

- 一般形式：

```
void  Obstinate(input x, solution y)
{
   // 反复调用拉斯维加斯算法LV(x,y)，直到找到问题的一个解y
   success=false;
   while (!success) success=LV(x, y);
}
```

- 设p(x)是对输入x调用拉斯维加斯算法获得问题的一个解的概率，一个正确的拉斯维加斯算法应该对所有输入x均有p(x)>0。
- 设t(x)是算法Obstinate找到具体实例x的一个解所需的平均时间，s(x)和e(x)分别是算法对于具体实例x求解成功或求解失败所需的平均时间，则有：

解此方程可得：

$$t(x) = p(x)s(x) + (1 - p(x))(e(x) + t(x))$$

- 能显著改进算法的时间复杂性！

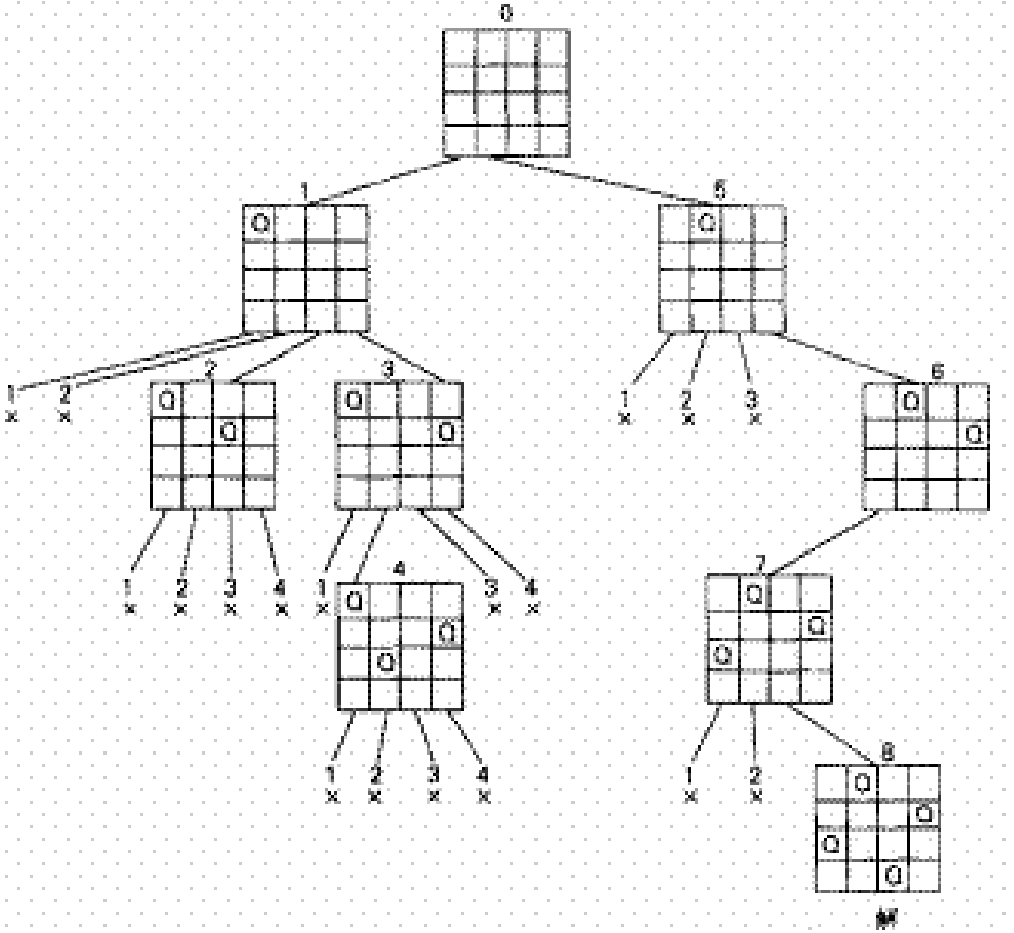
$$t(x) = s(x) + \frac{1 - p(x)}{p(x)} e(x)$$

LV

* 1、八皇后问题

①完全回溯法

用回溯法解n皇后问题
n叉树表示解空间。可行性
剪去不满足行、列和斜线
如右图（四皇后）：



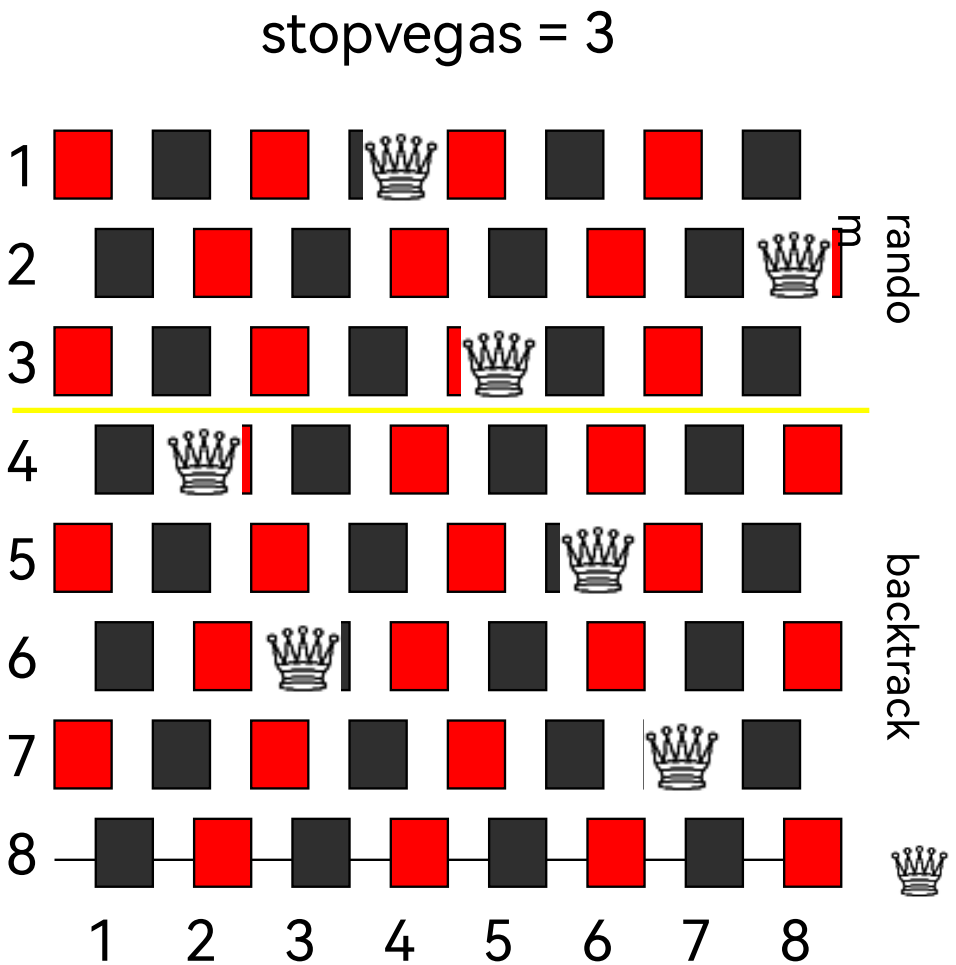
②完全LV算法

* 对于八皇后问题的任何一个解而言，每一个皇后在棋盘上的位置无任何规律，不具有系统性，而更像是随机放置的.
* 由此想到可采用LV算法，满足新放置的皇后与已放置的皇后互不攻击前提下，在棋盘上相继的各行中随机地放置皇后，直至8个皇后均已相容地放置好，或已没有下一皇后可放置位置时为止.

7.5 拉斯维加斯型概率算法

③回溯算法与lv算法混合使用

* 先在棋盘的前若干行中随机地放置皇后，（即设定stopvegas的值）然后在后继行中用回溯法继续搜索，直至找到一个解或宣告失败.
* 随机放置的皇后越多，后继回溯搜索所需的时间就越少，但失败的概率就越大.



运行算法

运行结果

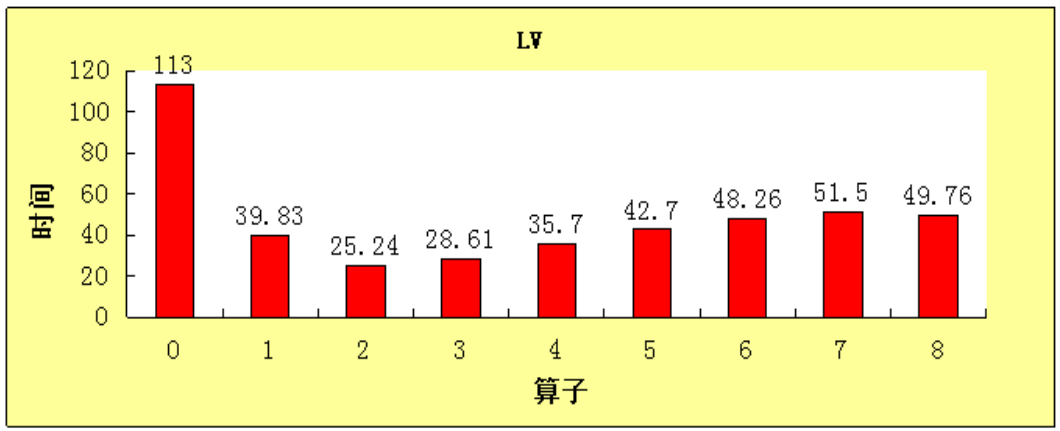
stopVegas	p	s	e	t
0	1.000000	113.000000	0.000000	113.000000
1	1.000000	38.165001	0.000000	38.165001
2	0.854701	21.545000	39.852940	28.320000
3	0.478469	12.515000	15.784404	29.720001
4	0.235571	9.305000	8.884438	38.134998
5	0.138408	8.390000	7.165462	52.995003
6	0.151057	8.035000	6.662811	45.480000
7	0.138504	8.000000	6.315917	47.285000
8	0.129955	8.000000	6.017177	48.285000

完全回溯法

LV和回溯混合效率最高的情况

完全LV算法

分析：



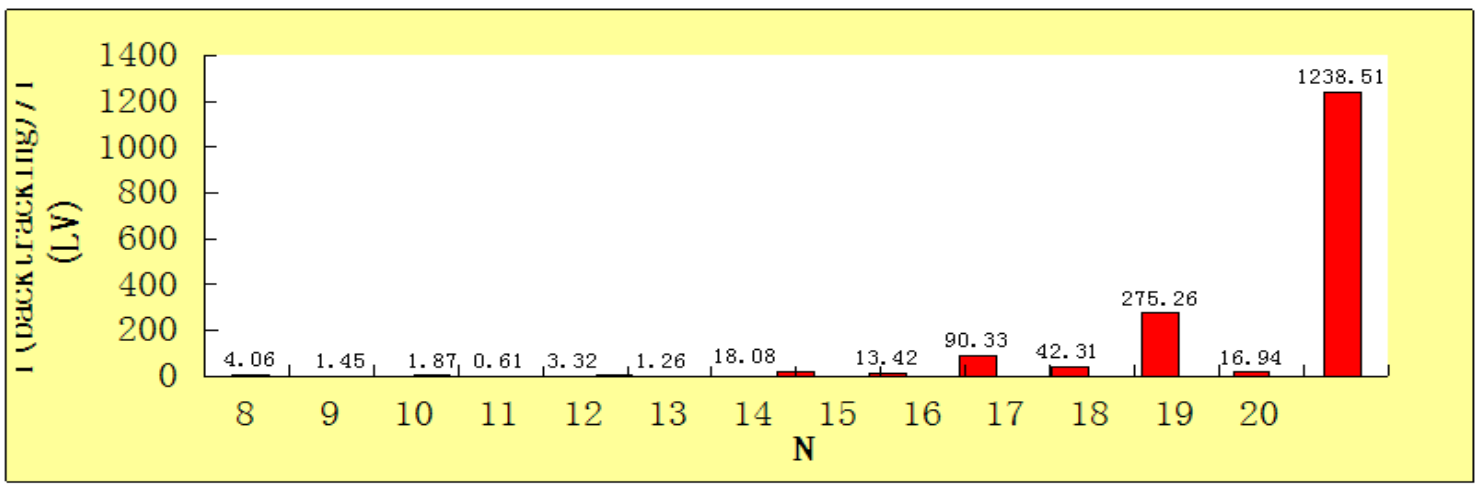
①完全回溯法比LV算法在解八皇后问题时效率低得多；

②LV算法和回溯的混合使用有利于提高解八皇后的效率。

LV算法具体应用分析

2、N皇后问题

* 下面是n取不同值时效率对比情况：



随着n值的增加，与回溯法相比，使用LV算法的高效率性逐渐明显。

将拉斯维加斯与回溯法相结合求解n皇后问题，先在棋盘的若干行中随机地放置皇后，然后在后继行中用回溯法继续放置，直到找到一个解或宣告失败。随机放置的皇后越多，后继回溯搜索所需的时间就越少，但失败的概率也就越大。

下表给出用拉斯维加斯回溯算法解八皇后问题时，对于不同的stopVegas值，算法成功的概率p，一次成功搜索访问的结点数平均值s，一次不成功搜索访问的节点数平均值e，以及反复调用算法使得最终找到一个解所访问的节点数的平均值t=s+(1-p)e/p。

```
#include <iostream>
#include <cmath>
#include "RandomNumber.h“
using namespace std;
class Queen{
    friend bool nQueen(int);
    private:
    bool Place(int k);//测试皇后k置于第x[k]列的合法性
    bool Backtrack(int t);//解n后问题的回溯法
    bool QueensLV(int stopVegas);//随机放置n个皇后拉斯维加斯算法
    int n,*x,*y;};
```

```
bool Queen::Place(int k)
{//测试皇后k置于第x[k]列的合法性
    for(int j=1;j<k;j++)
        if((abs(k-j)==abs(x[j]-x[k]))||(x[j]==x[k]))
            return false;
    return true;}
bool Queen::Backtrack(int t){//解n后问题的回溯
法
   if(t>n){
        for(int i=1;i<=n;i++)
            y[i]=x[i];
        return true;    }
    else
        for(int i=1;i<=n;i++)
        {   x[t]=i;
            if(Place(t)&&Backtrack(t+1))
                return true;
       }
        return false;}
```

```
bool Queen::QueensLV(int stopVegas)
{//随机放置n个皇后拉斯维加斯算法
   RandomNumber rnd;    int k=1;//随机数产生器
   int count=1;    //1<=stopVegas<=n表示允许随机放置的皇后数
   while((k<=stopVegas)&&(count>0))
   {       count=0;
      for(int i=1;i<=n;i++)
      {   x[k]=i;
         if(Place(k))//x里记录了当前棋盘布局
            y[count++]=i;
      }//y里记录了第k行时的所有可选项
      if(count>0)
         x[k++]=y[rnd.Random(count)];//随机在y的选项里抽一个
   }
   return (count>0);//count>0表示放置成功}
```

```
bool nQueen(int n)
{   //与回溯法相结合的解n后问题的拉斯维加斯算法
    Queen X;   //初始化X
    X.n=n;
    int *p=new int[n+1];
    int *q=new int[n+1];
    for(int i=0;i<=n;i++)
    {   p[i]=0;
        q[i]=0;  }
    X.y=p;   X.x=q;
    int stop=2;
    if(n>15)
        stop=n-15;
    bool found=false;
    while(!X.QueensLV(stop));   //算法的回溯搜索部分
    if(X.Backtrack(stop+1))
    {  for(int i=1;i<=n;i++)
            cout<<p[i]<<" ";
        found=true;   }
    cout<<endl;   delete []p;   delete []q;   return found;}
```

```
int main()
{
if(!nQueen(8))
{
cout<<"搜索不成功！！"<<endl;
}
system("pause");
return 0;
}
```

特点

- Monte Carlo算法总能求得问题实例的解；
- Monte Carlo算法可能返回不正确的解。
  - 算法求得正确解的概率依赖于算法所用的时间，算法所用时间越多，得到正确解的概率就越高。
- 算法的参数：
  - 描述问题实例的输入参数I；
  - 描述错误解可接受概率的参数ε。
- 用于求问题的准确解。

- 主元素问题。

例7-5:设T[1]~T[n]是一个含有n个元素的数组。当T中取值等于x的元素个数大于n/2时，称x是T的主元素。试设计算法找出T的主元素。

- 蒙特卡罗型概率算法求解思想：随机地选择数组中的一个元素T[i]进行统计，如果该元素出现的次数大于$n/2$，则该元素就是数组的主元素，算法返回true；否则随机选择的这个元素T[i]不是主元素，算法返回false。算法如下：

```
bool MC_Majority( Type A[], int n, Type &x )
{  int i, j, sum;
   i = Random(n);
   x = A[i]; sum = 0;
   for ( j = 0; j < n; j++ )
     if ( x == A[j] ) sum ++;
   if ( sum > n / 2 )
     return(true);
   else return(false);
}
```

算法的输出结论符合问题实例的实际情况吗?

如， 数组T[7]={3, 2, 3, 2, 3, 3, 5}。

当i= Random(7)=0、2、4、5时，算法返回正确的结果;

当i= Random(7)=1、3、6等时，算法返回错误的结果。

□“主元素问题”Monte Carlo算法的分析：

根据算法思想，算法返回True的概率>1/2，而返回false的概率≤1/2。

∴算法返回错误结论的概率≤1/2

若执行MC_Majority算法k次，则算法返回错误结论的概率≤2^{-k}。

令：算法返回错误结论的概率为ε，

由$2^{-k}=\varepsilon$，有：$k=\log(1/\varepsilon)$。

即，在重复执行k次Monte Carlo算法中，只要有一次返回true，就可以判断存在主元素，且算法返回错误结论的概率小于ε。

□ 改进“主元素问题”的Monte Carlo算法：

```
bool MC_Majority_2( int n, Type &x, double ε)
{ int i, j, k, m, sum; bool flag = false;
  k = log(1 /ε);
  for ( m = 1; m <= k; m++ ) {
    i = Random(n);
    x = A[i]; sum = 0;
    for ( j = 0; j < n; j++ )
      if ( x == A[j] ) sum++;
    if ( sum > n / 2 ) { flag = true; break; }
  }
  return(flag);
}
```

算法时间复杂度： $O(n\log(1/\varepsilon))$

本章小结

- 概率算法把随机性的选择注入到算法中，在算法执行某些步骤时，可以随机地选择下一步该如何进行。
- 允许结果以较小的概率出现错误，并以此为代价，获得算法运行时间的大幅度减少。
- 概率算法设计中采用线性同余法产生伪随机数。
- 设计概率算法的步骤如下:
  - 对给定问题设计一个确定算法A;
  - 分析算法A的各个步骤及每一步骤的有关操作，以选定随机化的入口和随机化方式，从而将算法A随机化，得到概率算法A’;
  - 论证A’是一个好的算法，即证明：A’的正确性和A’比A更好。

- 舍伍德型概率算法:
  - 总能求得问题的一个正确解;
  - 能消除算法最坏特性与平均特性之间的差别。
- 拉斯维加斯型概率算法:
  - 有时找不到问题实例的解;
  - 所能找到的解是正确的;
  - 算法的返回类型为bool;
  - 有两个参数:
    - 算法的输入
    - 当算法运行成功时保存问题的解
  - 能显著改进算法的时间复杂性!

本章小结

- 蒙特卡罗型概率算法:
  - 总能求得问题实例的解;
  - 可能返回不正确的解;
  - 用于求问题的准确解。